



Mobile and Autonomous Robots (UE23CS343BB7)

Jackfruit Problem (Mini-Project Report)

Project Title:

Autonomous Fire Detection and Interception utilizing Monocular Vision

Student Name(s) & SRN(s):

1. Mahika Neranjen	PES1UG23CS340
2. Surabhi Venkatesha	PES1UG23CS919
3. Brinda S	PES1UG23CS150
4. B M Krupa	PES1UG23CS133

Instructor:

Dr. Adithya Balasubramanyam



1. Introduction

- **Problem Statement:** Autonomous mobile robots require the ability to perceive their environment, navigate toward a target, and track their own movement — all without human intervention. Achieving this using low-cost, accessible hardware (a smartphone camera and an Arduino-based motor controller) is a practical challenge that sits at the intersection of computer vision, robotics, and embedded systems.
- **Importance:** In robotics and automation, closed-loop visual feedback systems are fundamental to enabling robots to operate in dynamic environments. The ability to detect a target, make steering decisions, and simultaneously track position forms the basis of more advanced autonomous systems. Building this from affordable components demonstrates real-world feasibility for low-budget robotics applications.
- **Overview of Solution:** This project implements a visual target-tracking and odometry system using a custom ROS 2 node written in Python. The node reads frames from a Wi-Fi-connected smartphone camera, isolates a high-intensity target (such as a flame) using dynamic HSV color segmentation, makes steering decisions, and sends motor commands to an Arduino over USB serial. Simultaneously, sparse optical flow is used to estimate the robot's movement and publish a qualitative path to ROS 2 for visualization in RViz2.

2. Objective

- To build an autonomous robot capable of detecting and navigating toward a specific visual signature (calibrated for flame detection) using a smartphone camera.
- To implement lightweight visual odometry using Lucas-Kanade optical flow for real-time path estimation.
- To publish the robot's estimated trajectory to ROS 2 and visualize it dynamically in RViz2.
- To control a 4WD/differential drive robot via Arduino over USB serial, incorporating fault-tolerant features like a safety deadman switch.

Expected Outcomes:

- The robot autonomously searches for, steers toward, and stops at a designated high-intensity target (flame).
- A live, qualitative path trace of the robot's movement is published on the `/robot_path` ROS 2 topic and visualized in RViz2.

Scope: The project is limited to indoor operation, single-target tracking based on HSV calibration, Wi-Fi camera streaming, and Arduino-based motor control.

Due to hardware constraints (monocular forward-facing camera), metric 3D map building is out of scope; odometry provides a qualitative 2D heuristic.

3. Methodology

The system operates as a continuous perception-decision-action loop. Each frame from the smartphone camera is processed for two parallel tasks: visual odometry (to estimate and publish the robot's path) and color-based target detection (to determine the motor command). The resulting command character is sent over USB serial to the Arduino, which drives the motors accordingly.

3.1 Algorithms

A. Lucas-Kanade Optical Flow (Visual Odometry)

Shi-Tomasi corner detection (**cv2.goodFeaturesToTrack**) identifies trackable feature points in the grayscale frame. The Lucas-Kanade pyramidal optical flow algorithm (**cv2.calcOpticalFlowPyrLK**) then tracks these points between consecutive frames. The mean displacement of successfully tracked points is computed and scaled by a small factor (0.01) to estimate the robot's change in position. This estimate (**dx**, **dy**) is accumulated into a global (**global_x**, **global_y**) coordinate, which is published as a **PoseStamped** message appended to a **nav_msgs/Path** on the **/robot_path** topic.

This algorithm was deliberately chosen over dense optical flow or ORB-SLAM to fit a strict computational budget. Running heavy feature-matching over a Wi-Fi IP stream would bottleneck the CPU and drop the frame rate, causing the robot's steering reflexes to fail. Lucas-Kanade provides a computationally cheap 'pseudo-odometry' visualization while maintaining the real-time processing speed required to intercept the target.

B. HSV Color Segmentation (Target Detection)

Each frame is converted from BGR to the HSV color space, which isolates brightness (Value) from color (Hue), making it highly resistant to dynamic room lighting. A binary mask is generated using **cv2.inRange**. For fire detection, the bounds are calibrated to capture extreme brightness and orange/yellow hues (e.g., High Value, Low-to-Medium Saturation). Contours are extracted from this mask. Any contour with an area greater than 1,000 pixels passes the low-pass noise filter and is considered a valid target. The bounding box and centroid of the largest qualifying contour are then computed.

C. Proportional Steering Logic

Based on the horizontal position of the detected target's center relative to the frame center (640px wide), one of four commands is issued:

- If the target center is more than 100px to the left → turn left (**a**)
- If the target center is more than 100px to the right → turn right (**d**)
- If centered → drive forward (**w**)

- If the contour area exceeds 80,000px (target is very close) → stop (**x**)
- If no target is found → spin right to search (**d**)

D. Arduino Motor Control (Deadman Switch)

The Arduino receives single-character commands (**w, a, d, s, x**) over serial at 9600 baud. It drives a differential drive robot through a dual-channel motor driver using PWM signals on pins 5, 6, 9, and 10, with enable pins on 7, 8, 11, and 12. A safety deadman switch halts both motors if no command is received for 1000ms, preventing runaway motion if the serial connection drops.

3.2 Tools Used

Middleware & Software Environments:

- **ROS 2 (Robot Operating System):** The primary middleware framework used for node management, inter-process communication, and data broadcasting.
- **RViz2:** The standard ROS 2 3D visualization tool, used to graphically plot the qualitative odometry trajectory (`/robot_path`) in real-time.
- **Ubuntu Linux:** The host operating system required for native ROS 2 execution and serial port management (`/dev/ttyACM0`).
- **Python 3:** Core language used to develop the vision and control **ROS 2 node**.
- **Arduino IDE:** Used to compile and flash the C++ hardware-level PWM control and deadman switch logic to the microcontroller.

Python Libraries & Packages (Dependencies):

- **OpenCV (`cv2`):** Handled all core Computer Vision tasks, including the Lucas-Kanade optical flow (`calcOpticalFlowPyrLK`), Shi-Tomasi feature extraction, HSV color space conversions, and contour generation.
- **NumPy (`numpy`):** Used for high-speed array manipulation, specifically computing the mean displacement vectors (`$dx, dy$`) of tracked optical flow features and defining HSV threshold arrays.
- **PySerial (`serial`):** Provided the USB serial bridge to transmit encoded UTF-8 ASCII characters to the Arduino, complete with exception-handling for hardware disconnects.
- **rclpy:** The official ROS 2 Python client library used to initialize the node and manage the publish/subscribe lifecycle.
- **ROS 2 Message Interfaces:** Specifically utilized `nav_msgs.msg.Path` and `geometry_msgs.msg.PoseStamped` to properly structure the coordinate transform data for RViz2.

Hardware Architecture:

- **Processing Unit:** Standard laptop running the Python/ROS 2 master node (Edge Computing).
- **Microcontroller:** Arduino (acting as the low-level motor hardware driver).
- **Actuation:** Differential drive chassis powered by four DC motors via a dual-channel motor driver (IBT-2) utilizing PWM for speed control and digital pins for direction.
- **Perception Sensor:** Smartphone camera acting as a wireless IP Camera, streaming video data over a local Wi-Fi TCP/IP bridge.

4. Outcome

The system successfully achieved autonomous target acquisition. The dual-pipeline architecture allowed the robot to simultaneously hunt the target and map its trajectory without dropping below the required frame rate for real-time motor control.

4.1 Result Screenshots



Figure 1: The autonomous robot successfully completes a target acquisition sequence. Utilizing the smartphone camera for visual servoing, the robot navigated toward the designated green target and successfully engaged the software-based proximity halt once the object's contour area exceeded the predefined safety threshold.

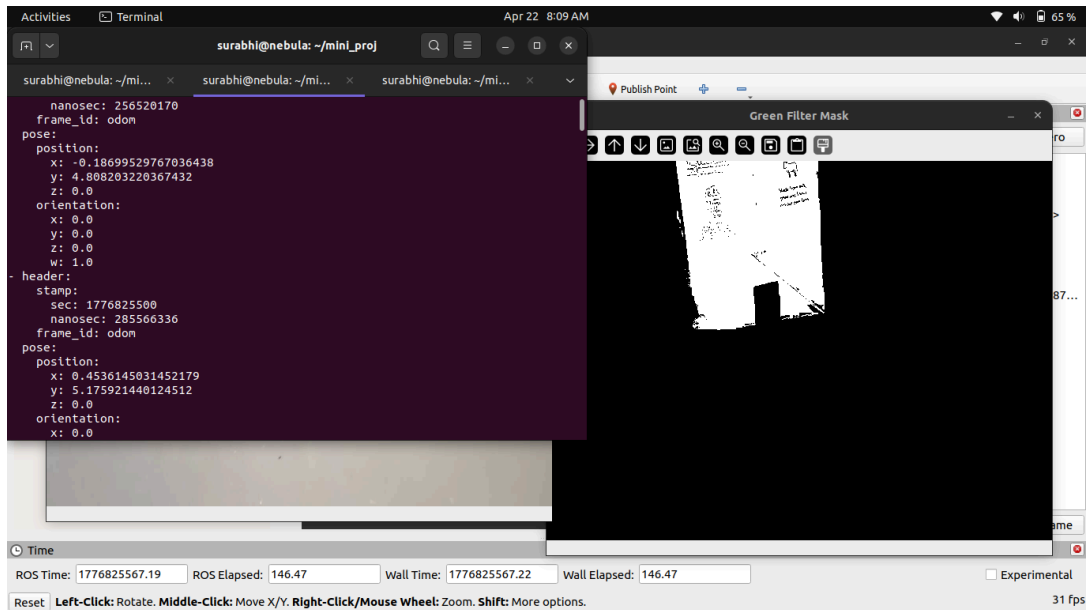


Figure 2: Echoing the topic /robot_path in the terminal: it is publishing x and y coordinates along with the corresponding timestamp so that Rviz can plot the path

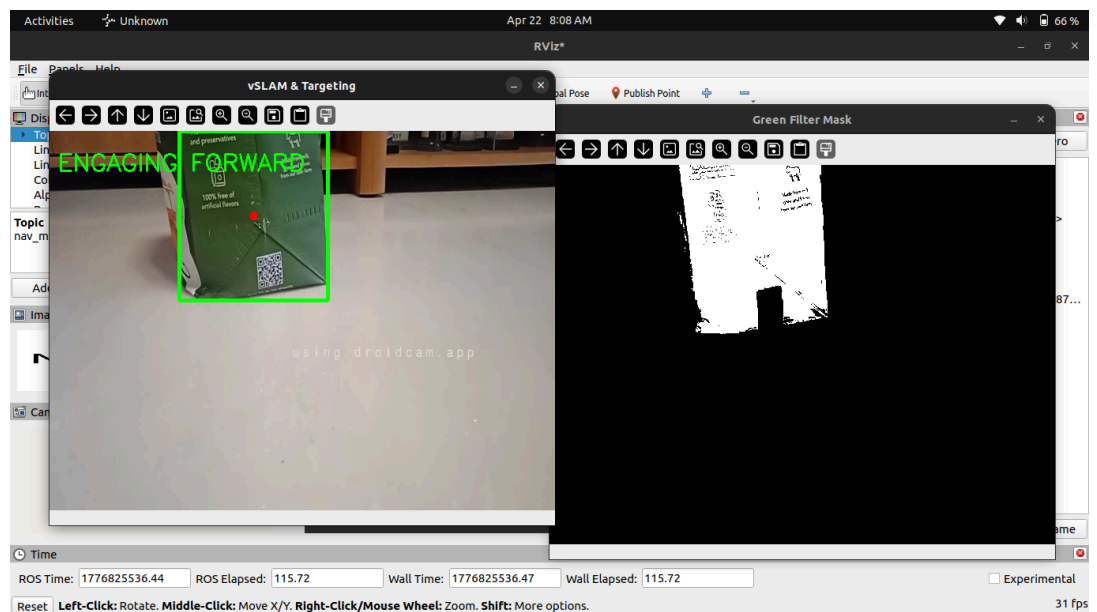


Figure 3: Robot camera view detecting the object and the command it is issuing the the Arduino board (left) and the HSV filter mask for the detected object (right)

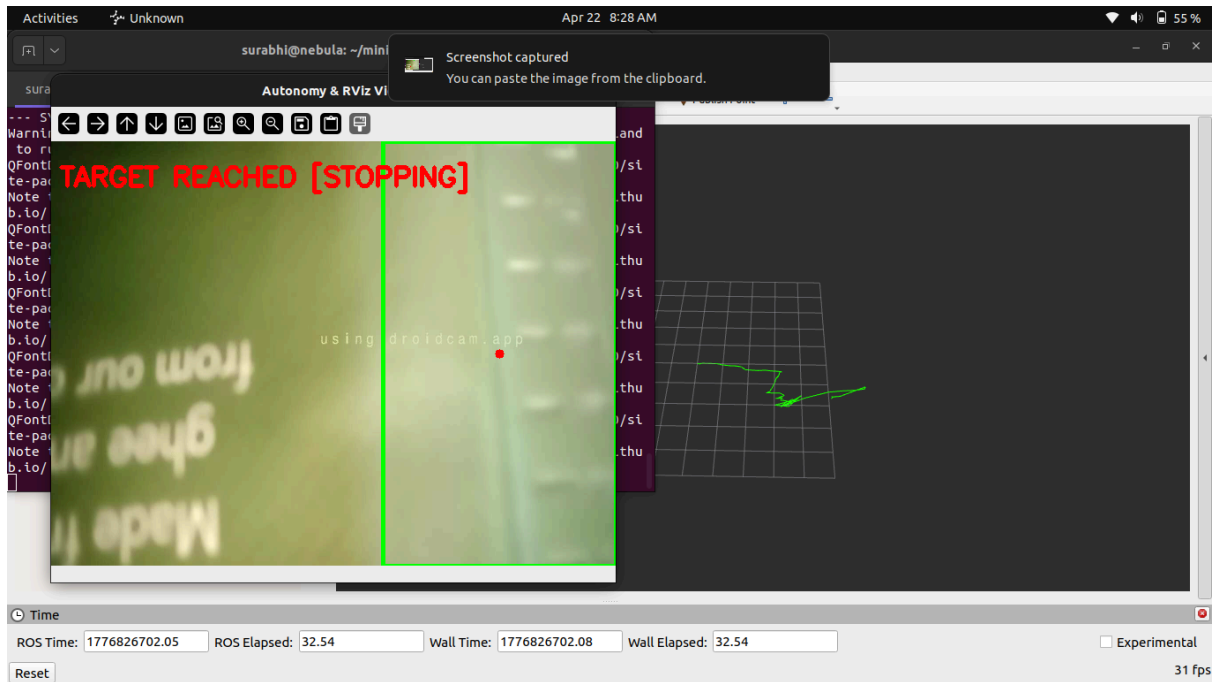


Figure 4: Successful mission completion. The left window confirms the target has been acquired and the drivetrain halted via the area-based safety threshold. Simultaneously, the right window displays the final accumulated trajectory of the approach generated by the Lucas-Kanade odometry node.

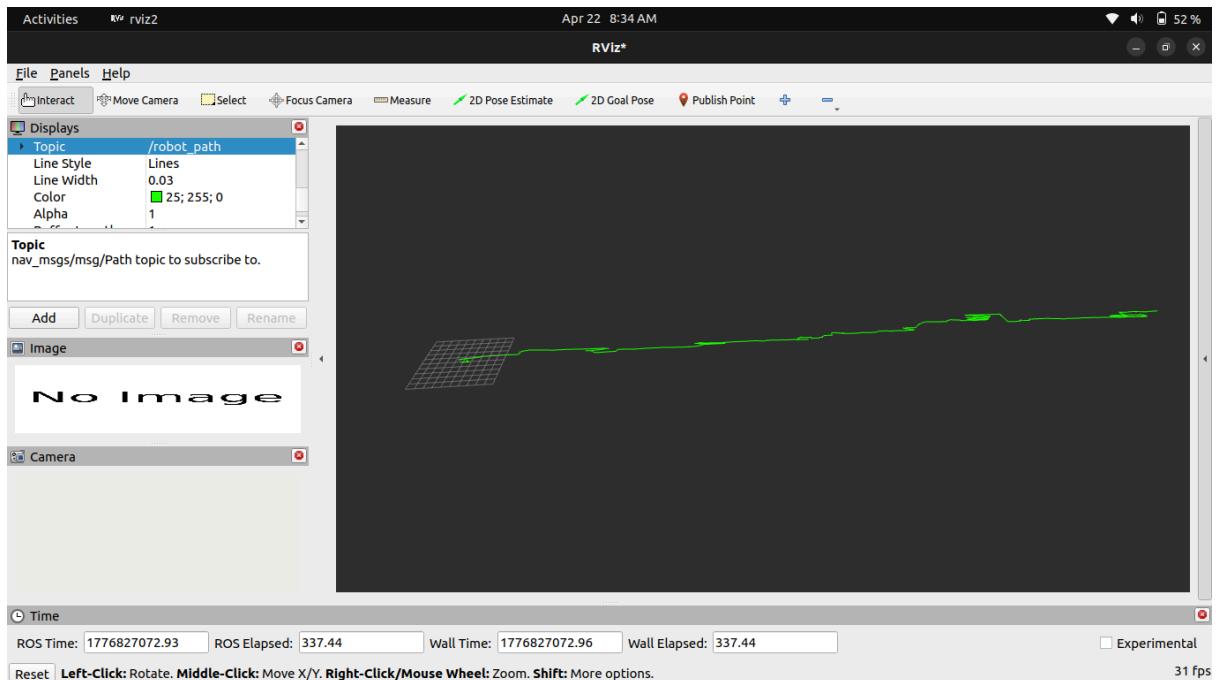


Figure 5: RViz2 visualization of the `/robot_path` topic generated via Lucas-Kanade optical flow. The visible angular deviations and jagged trajectory perfectly demonstrate the expected odometry drift, a known mathematical constraint of utilizing a monocular, forward-facing camera without loop-closure integration.

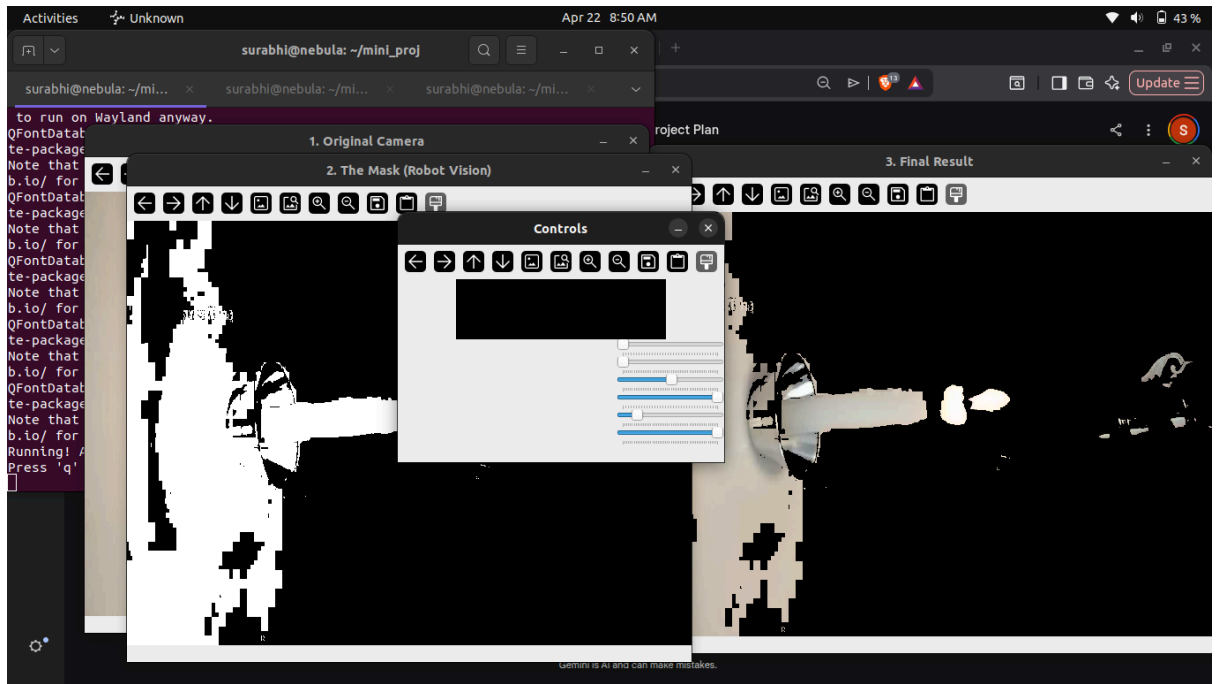


Figure 6: The custom HSV Calibration diagnostic tool in operation. We utilized this interface to dynamically tune the Hue, Saturation, and Value thresholds for the flame.

4.2 Video Demonstration

Drive Link:

https://drive.google.com/drive/folders/1_3WarQzfrUipXNnUNwEvqT5Q8Ap_Xdy_?usp=sharing

4.3 GitHub Repository

GitHub Link: <https://github.com/surabhi1828/autonomous-detection-4wd-robot>

5. Conclusion

Key Learnings:

- Integrating computer vision with real-time hardware control requires careful attention to latency; the 60ms per-frame loop had to remain fast enough to keep motor commands responsive.
- Optical flow provides a usable odometry signal from a monocular camera but is noisy and sensitive to lighting changes, which is why a small scale factor (0.01) was applied to suppress jitter.
- The deadman switch on the Arduino proved essential — without it, motors would continue running if the USB connection dropped or the ROS 2 control node crashed.
- HSV color segmentation is sensitive to calibration; the green detection bounds had to be tuned for the specific lighting environment.

Challenges Faced:

- **Epipolar Constraints of a Monocular Camera:** We discovered that running Lucas-Kanade on a forward-facing camera causes optical flow vectors to expand outward rather than sliding linearly. We had to optimize the scale factor to adapt this 2D expansion into a usable 2D odometry heuristic without requiring a dedicated downward-facing sensor.
- Maintaining a stable Wi-Fi camera stream while simultaneously running optical flow and serial communication required careful resource management.
- USB serial drops during operation required implementing on-the-fly reconnection logic in the ROS2 Python control node.
- Tuning the steering deadzone ($\pm 100\text{px}$) was necessary to prevent the robot from stuttering when the target was near the center.